

DOCUMENTAZIONE PROGETTO

PRONTO SOCCORSO INTELLIGENTE

Gruppo di lavoro

Valeria Agostinacchio - matricola758264 - v.agostinacchio1@studenti.uniba.it

Repository GitHub

<https://github.com/Vastrusa/ProgettoICON/tree/main>

Anno Accademico 2024–2025

INDICE

1.Introduzione

2.Architettura del Sistema

3. Ontologia

4. Knowledge Base (KB)

5. Fatti e Regole della KB

6. Query della KB

7. Aggiornamento della KB

8. Apprendimento Supervisionato

Data Cleaning

Scelta del modello

Valutazione

Conclusioni

9. Algoritmo di Path Finding (A\)*

Sommario

Strumenti utilizzati

Decisioni di progetto

Valutazione

Conclusioni

10. Rete Bayesiana

Sommario

Strumenti utilizzati

Decisioni di progetto

Valutazione

Conclusioni

11. Valutazione Complessiva

12. Discussione e Limiti

13. Conclusioni

1 - Introduzione

Il progetto *Pronto Soccorso Intelligente* affronta il problema dell'assegnazione del codice di triage attraverso un approccio ibrido che combina conoscenza clinica formalizzata e apprendimento supervisionato. L'obiettivo è riprodurre, in forma computazionale, il processo decisionale tipico del triage ospedaliero, integrando da un lato regole cliniche esplicite e verificabili, e dall'altro la capacità predittiva di un modello statistico addestrato su esempi reali o simulati.

Il dominio del triage si presta in modo naturale a questa integrazione: molte decisioni si basano su sintomi, parametri vitali e soglie numeriche chiaramente definite, che possono essere modellate tramite un'ontologia e regole inferenziali. Allo stesso tempo, la variabilità dei quadri clinici e la presenza di pattern non immediatamente codificabili rendono utile l'impiego di un modello di machine learning, capace di generalizzare e individuare correlazioni non esplicitamente rappresentate nelle regole.

L'unione di questi due approcci consente di ottenere un sistema più robusto, trasparente e flessibile, in grado di supportare il processo di triage attraverso inferenze logiche e predizioni data-driven.

2 - ARCHITETTURA DEL SISTEMA DI TRIAGE

Architettura del Sistema

L'architettura del sistema di triage è stata progettata per integrare in modo coerente e modulare componenti eterogenei — simbolici, semantici, probabilistici e statistici — all'interno di un'unica pipeline decisionale. L'obiettivo è combinare la trasparenza delle regole cliniche con la capacità predittiva dei modelli di apprendimento, ottenendo un sistema robusto, interpretabile e capace di gestire casi complessi.

L'intero sistema è organizzato secondo un'architettura **ibrida multilivello**, in cui ogni modulo contribuisce con una prospettiva diversa sulla condizione clinica del paziente. La comunicazione tra i moduli è orchestrata dal backend Python, che funge da punto di integrazione e da motore decisionale finale.

Struttura generale

L'architettura è composta da cinque livelli principali:

1. **Interfaccia utente**
2. **Backend Python**
3. **Motori simbolici e semantici**

- **Ontologia OWL**
- **Knowledge Base Prolog**

4. **Modelli probabilistici e statistici**

- **Random Forest e Regressione Logistica**
- **Naive Bayes**
- **Rete Bayesiana**
- **A***

5. **Modulo di integrazione e validazione**

Flusso dei dati

Il processo decisionale segue una pipeline ben definita:

1. **Input del paziente** Sintomi e parametri vitali vengono raccolti dal frontend.
2. **Normalizzazione e pre-processing** Il backend converte i dati in un formato compatibile con tutti i moduli.
3. **Esecuzione parallela dei modelli**
 - OWL inferisce il codice tramite regole semantiche.
 - Prolog applica regole logiche deterministiche.
 - ML predice il codice tramite apprendimento supervisionato.
 - La Rete Bayesiana calcola la distribuzione probabilistica dei codici.
 - A* simula l'evoluzione clinica e individua il percorso a costo minimo.
4. **Raccolta e confronto dei risultati** Ogni modulo restituisce un codice di triage o una distribuzione di probabilità.
5. **Decisione finale** Il sistema applica una logica di fusione che tiene conto di:
 - coerenza tra modelli
 - gravità massima rilevata
 - predizioni probabilistiche
 - evoluzione clinica prevista da A*
6. **Output all'utente**

3 - Ontologie

L'ontologia ProntoSoccorso.owl costituisce il modello formale attraverso cui viene rappresentata in modo strutturato, coerente e computabile la conoscenza clinica necessaria al processo di triage. È stata sviluppata in OWL 2, linguaggio standard per la modellazione semantica, e consente di descrivere concetti, relazioni e vincoli del dominio sanitario in modo tale che un reasoner possa inferire automaticamente il codice di priorità più appropriato per ciascun paziente.

L'ontologia è organizzata come una gerarchia di classi e proprietà che modellano sintomi, parametri vitali, categorie di triage ed eventuali esami diagnostici. Ogni concetto è identificato da un URI univoco e collegato agli altri tramite relazioni semantiche, permettendo di costruire una base di conoscenza navigabile, interrogabile e soprattutto interpretabile da un motore inferenziale.

La struttura dell'ontologia è radicata in owl:Thing, da cui derivano tutte le altre classi. Le principali entità modellate sono:

Paziente: rappresenta l'individuo che accede al Pronto Soccorso e costituisce il nodo centrale del processo di triage.

Sintomo: descrive una manifestazione clinica rilevante (es. dolore toracico, difficoltà respiratoria, febbre, raffreddore). Ogni sintomo è modellato come sottoclasse di Sintomo e istanziato come individuo quando presente in un paziente.

ParametroVitale: rappresenta un valore misurabile, come saturazione o frequenza cardiaca. Ogni parametro è suddiviso in sottoclassi che rappresentano intervalli clinicamente significativi (es. SaturazioneBassa, FrequenzaAlta), consentendo al reasoner di applicare regole basate su soglie.

CodiceTriage: identifica la categoria di priorità clinica (Rosso, Giallo, Verde, Bianco), modellata come gerarchia di classi mutuamente disgiunte.

Le relazioni tra gli elementi sono espresse tramite object properties, che definiscono i collegamenti semantici tra le classi:

presenzaSintomo collega un paziente ai sintomi rilevati.

haParametroVitale associa un paziente ai suoi parametri vitali.

assegnatoCodiceTriage collega un paziente al codice di triage inferito dal reasoner.

Le informazioni numeriche, come i valori dei parametri vitali, sono rappresentate tramite data properties, tra cui valoreParametro, che consente l'applicazione di regole SWRL basate su soglie numeriche (ad esempio saturazione < 90, frequenza cardiaca > 120, ecc.).

L'ontologia è strutturata come un grafo semantico: ogni individuo è un nodo, e le relazioni tra sintomi, parametri vitali e codici di triage costituiscono gli archi. Questa rappresentazione permette di navigare facilmente la conoscenza clinica e di applicare inferenze logiche tramite il reasoner Pellet, che elabora le regole SWRL e le relazioni ontologiche per dedurre automaticamente il codice di triage più appropriato.

Durante lo sviluppo è stata posta particolare attenzione alla coerenza della gerarchia, evitando cicli di ereditarietà e ambiguità semantiche che avrebbero impedito al reasoner di operare correttamente. La distinzione rigorosa tra classi (es. DifficoltaRespiratoria) e individui (es. DifficoltaRespiratoria1) ha garantito la piena compatibilità con OWL 2 DL e il corretto funzionamento dell'inferenza.

L'ontologia, integrata con il backend Python tramite Owlready2, costituisce quindi un componente fondamentale del sistema: fornisce una base di conoscenza esplicita, verificabile e riutilizzabile, capace di supportare decisioni cliniche coerenti e trasparenti.

4 - Knowledge Base (KB)

La Knowledge Base (KB) rappresenta il nucleo logico del sistema di triage, ed è implementata tramite **programmazione logica in Prolog**. La KB contiene fatti e regole che descrivono il comportamento clinico atteso in base ai sintomi e ai parametri vitali del paziente, consentendo al sistema di assegnare un codice di priorità secondo criteri deterministici e trasparenti.

A differenza dell'ontologia, che modella la conoscenza in forma dichiarativa e semantica, la KB Prolog fornisce un livello di ragionamento operativo basato su regole logiche. Questo permette di definire condizioni cliniche precise e di verificare rapidamente se un paziente soddisfa i criteri per un determinato codice di triage.

5 - Fatti e regole della KB

La KB è organizzata attorno a tre concetti principali:

- **Sintomi**
- **Parametri vitali**
- **Codici di triage**

Ogni paziente viene rappresentato tramite fatti che descrivono i valori osservati, mentre le regole determinano il codice di priorità in base alle condizioni cliniche.

Classe sintomo

I sintomi sono rappresentati tramite fatti del tipo:

sintomo(Paziente, TipoSintomo).

Esempi:

- sintomo(p1, difficoltà_respiratoria).
- sintomo(p2, dolore_toracico).
- sintomo(p3, raffreddore).

I sintomi più rilevanti per il triage sono modellati come categorie distinte:

- difficoltà respiratoria
- dolore toracico
- febbre
- trauma
- raffreddore

Alcuni sintomi sono considerati **critici** (es. difficoltà respiratoria), altri **moderati** (es. febbre), altri ancora **lievi** (es. raffreddore). Questa distinzione permette alle regole di assegnare correttamente il codice.

Classe parametro vitale

I parametri vitali sono rappresentati tramite fatti del tipo:

parametro(Paziente, TipoParametro, Valore).

Esempi:

- parametro(p1, saturazione, 85).
- parametro(p2, fc, 130).

I parametri gestiti sono:

- **saturazione** (SpO₂)
- **frequenza cardiaca** (FC)

Questi valori vengono confrontati con soglie cliniche per determinare la gravità della condizione.

Regole di assegnazione del codice di triage

La KB contiene regole che stabiliscono il codice di triage in base ai fatti disponibili. Le regole seguono una logica gerarchica:

Codice Rosso (emergenza)

Condizioni che richiedono intervento immediato:

codice(P, rosso) :- sintomo(P, difficoltà_respiratoria).

codice(P, rosso) :- parametro(P, saturazione, S), S < 90.

codice(P, rosso) :- parametro(P, fc, FC), FC > 140.

Codice Giallo (urgenza)

Condizioni potenzialmente critiche:

codice(P, giallo) :- parametro(P, fc, FC), FC > 120, FC =>140.

codice(P, giallo) :- sintomo(P, trauma).

Codice Verde (urgenza minore)

Sintomi moderati o parametri alterati ma non critici:

codice(P, verde) :-

\+ codice(P, rosso),

\+ codice(P, giallo),

(

sintomo(P, febbre);

sintomo(P, raffreddore);

parametro(P, saturazione, S), S >= 90, S < 95;

parametro(P, fc, FC), FC >= 100, FC <= 120

).

Codice Bianco (non urgenza)

Assenza di sintomi rilevanti e parametri nella norma:

codice(P, bianco) :-

```
\+ codice(P, rosso),
```

```
\+ codice(P, giallo),
```

```
\+ codice(P, verde).
```

6 - Query della KB

Le query Prolog permettono di interrogare la KB per ottenere informazioni cliniche o verificare condizioni.

Esempio: determinare il codice di un paziente

```
?- codice(p1, C).
```

```
C = rosso.
```

Esempio: verificare la presenza di un sintomo

```
?- sintomo(p2, dolore_toracico).
```

```
true.
```

Esempio: controllare parametri vitali

```
?- parametro(p3, saturazione, S).
```

```
S = 94.
```

Ruolo della KB nel sistema

La KB Prolog svolge tre funzioni fondamentali:

1. **Valutazione deterministica della gravità clinica** Le regole permettono di stabilire un codice di triage basato su criteri chiari e verificabili.
2. **Supporto alla coerenza del sistema** La KB funge da riferimento logico per confrontare le decisioni del modello ML e dell'ontologia.
3. **Trasparenza decisionale** Ogni decisione può essere spiegata risalendo alle regole che l'hanno generata.

7 - Aggiornamento della Knowledge Base (KB)

La Knowledge Base (KB) Prolog rappresenta il componente logico del sistema di triage e contiene le regole cliniche utilizzate per determinare il codice di priorità di un paziente. Inizialmente, la KB include esclusivamente le regole statiche relative ai sintomi e ai parametri vitali, senza alcuna informazione sui pazienti che accedono al sistema. L'aggiornamento della KB avviene dinamicamente ogni volta che un nuovo paziente viene inserito tramite l'interfaccia dell'applicazione.

Per gestire la KB è stata sviluppata una classe Python dedicata, che utilizza la libreria **pyswip** per interagire con Prolog. Questa classe consente di:

- inserire nuovi fatti relativi ai pazienti (sintomi e parametri vitali)

- interrogare la KB per ottenere il codice di triage
- rimuovere o aggiornare le informazioni quando un paziente viene eliminato o modificato

L'aggiornamento della KB avviene quindi in tempo reale, garantendo che il reasoner Prolog lavori sempre su una base di conoscenza aggiornata e coerente.

Funzioni principali di aggiornamento della KB

1. aggiungi_paziente()

Questa funzione inserisce nella KB i fatti relativi a un nuovo paziente. Per ogni paziente vengono aggiunti:

- un fatto per ciascun sintomo rilevato sintomo(Paziente, TipoSintomo).
- un fatto per ciascun parametro vitale parametro(Paziente, Parametro, Valore).

In questo modo, la KB acquisisce tutte le informazioni necessarie per applicare le regole cliniche.

2. calcola_codice()

Dopo l'inserimento dei fatti, la funzione interroga Prolog tramite:

`codice(Paziente, Codice).`

Il predicato restituisce il codice di triage determinato dalle regole della KB. Questo valore viene poi confrontato con il codice assegnato dal modello ML e con quello inferito dall'ontologia.

3. aggiorna_paziente()

Se un paziente viene modificato (ad esempio cambiando sintomi o parametri vitali), la KB deve essere aggiornata. La funzione:

- rimuove i fatti precedenti relativi al paziente
- inserisce i nuovi fatti aggiornati
- ricalcola il codice tramite Prolog

Questo garantisce che la KB rimanga sempre coerente con lo stato attuale del sistema.

4. rimuovi_paziente()

Quando un paziente viene eliminato dalla coda, la KB deve essere ripulita dai fatti associati. La funzione rimuove:

- tutti i fatti sintomo(Paziente, _)
- tutti i fatti parametro(Paziente, _, _)

In questo modo si evita l'accumulo di informazioni obsolete che potrebbero interferire con le inferenze successive.

Esempio di aggiornamento della KB

Supponiamo che venga inserito un paziente con:

- sintomo: febbre
- saturazione: 92

- frequenza cardiaca: 110

La KB viene aggiornata con:

sintomo(p123, febbre).

parametro(p123, saturazione, 92).

parametro(p123, fc, 110).

Eseguendo la query:

?- codice(p123, C).

Prolog risponde:

C = giallo.

Il codice viene quindi restituito al backend e confrontato con ML e OWL.

Ruolo dell'aggiornamento dinamico

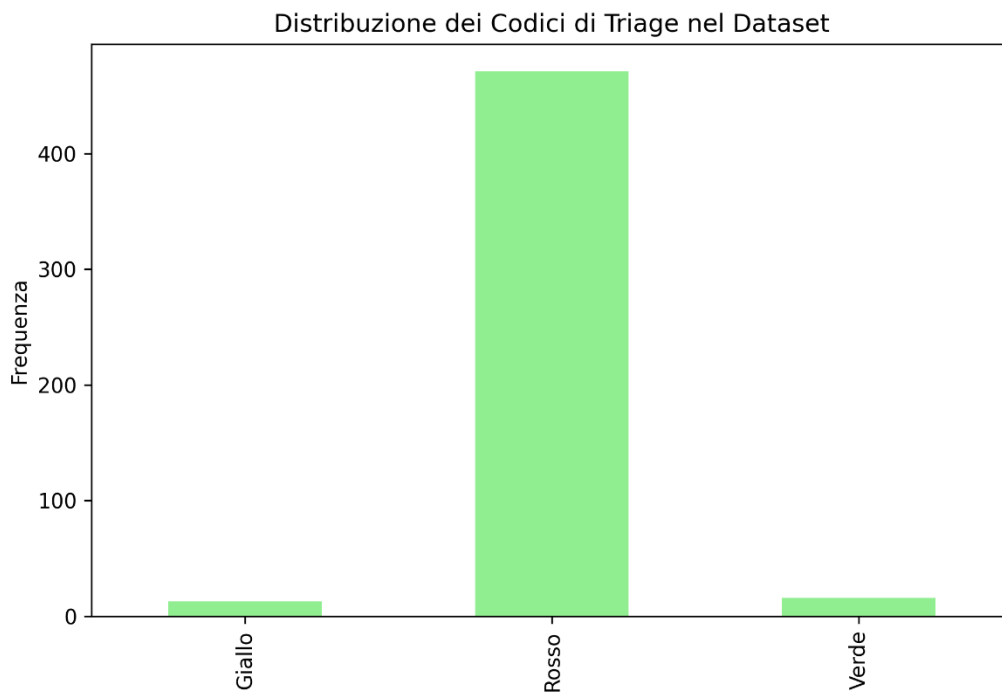
L'aggiornamento della KB permette al sistema di:

- mantenere una rappresentazione logica sempre aggiornata dei pazienti presenti
- applicare le regole cliniche in modo deterministico
- garantire coerenza tra i tre motori decisionali (Prolog, ML, Ontologia)
- supportare la trasparenza del processo di triage, poiché ogni decisione deriva da fatti e regole esplicite

8 – Apprendimento Supervisionato

Nel sistema di triage sviluppato, l'apprendimento supervisionato svolge un ruolo fondamentale nell'attribuzione del codice di priorità ai pazienti. L'obiettivo del modello è prevedere il codice di triage sulla base dei sintomi e dei parametri vitali rilevati al momento dell'accettazione, affiancando così la componente simbolica (Prolog) e quella semantica (ontologia OWL). Il modello supervisionato fornisce una valutazione statistica indipendente, utile per individuare pattern clinici complessi e per confrontare le decisioni degli altri moduli del sistema ibrido.

Il dataset utilizzato per l'addestramento è composto da esempi etichettati, ciascuno dei quali rappresenta un paziente descritto da un insieme di sintomi (febbre, dolore toracico, difficoltà respiratoria, trauma, raffreddore) e da parametri vitali (saturazione e frequenza cardiaca). A ogni caso clinico è associato un codice di triage, che costituisce la variabile target del modello.



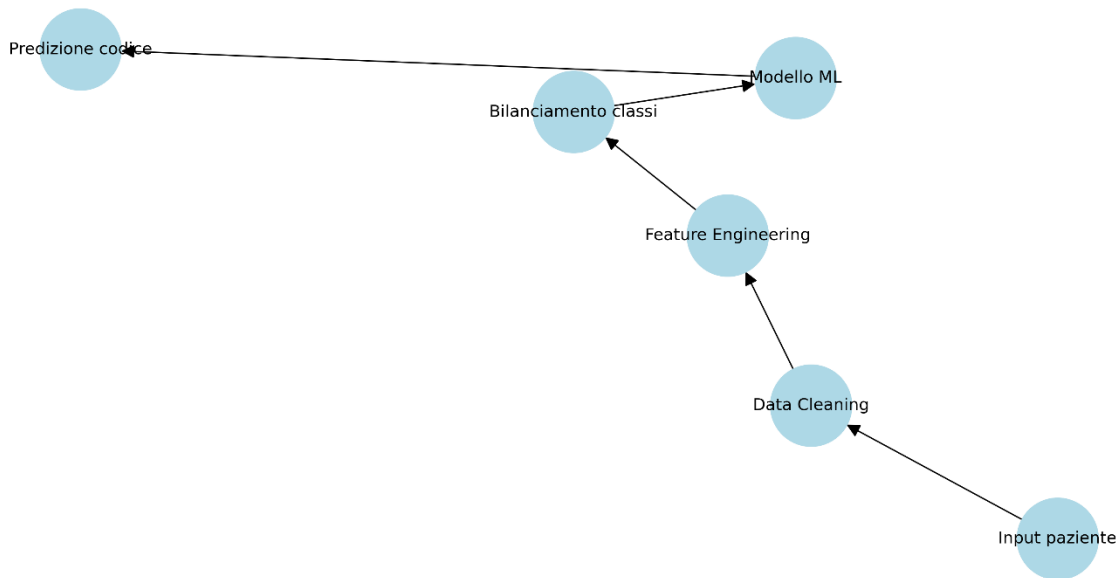
La distribuzione dei codici mostra una predominanza del codice Rosso, mentre le classi Verde e Giallo risultano meno rappresentate. Questa composizione riflette la natura del dataset utilizzato per l'addestramento, che enfatizza i casi clinicamente più critici. Tale sbilanciamento influisce sulle prestazioni del modello, che risulta particolarmente accurato nella classificazione dei casi gravi.

Data Cleaning

Prima dell'addestramento, il dataset è stato sottoposto a un processo di pulizia accurato. Sono stati rimossi i record incompleti o incoerenti, come quelli privi di sintomi o parametri vitali, o contenenti valori impossibili (ad esempio saturazione superiore al 100%). I parametri vitali numerici sono stati normalizzati tramite scaling, mentre i sintomi — essendo variabili categoriche — sono stati trasformati in variabili binarie tramite codifica one-hot.

Poiché la distribuzione delle classi non è uniforme, è stata applicata la pesatura automatica delle classi (`class_weight="balanced"`) durante l'addestramento, così da compensare la minore presenza delle classi Verde e Giallo e ridurre il rischio di bias verso la classe Rosso.

Pipeline del modello ML



Il diagramma illustra il flusso di elaborazione: dall'input del paziente fino alla predizione del codice, passando per le fasi di pulizia, codifica, bilanciamento e classificazione.

Scelta dei Modelli

Nel progetto sono stati adottati due modelli con caratteristiche complementari: Regressione Logistica e Random Forest.

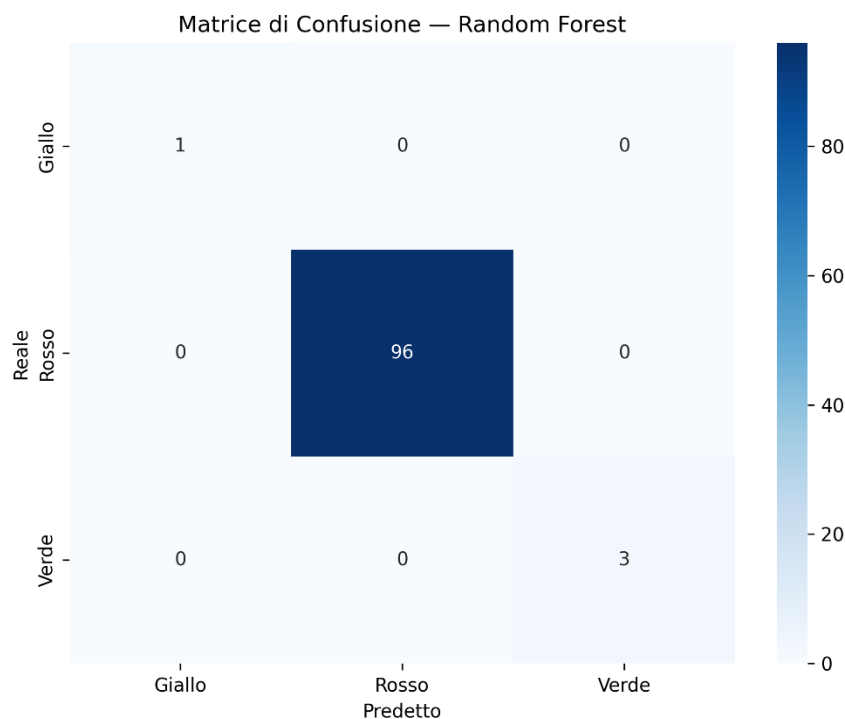
La Regressione Logistica è stata utilizzata come baseline interpretabile, con pesatura automatica delle classi per gestire eventuali squilibri residui e analizzare il contributo delle singole feature.

La Random Forest, invece, è stata scelta come modello operativo finale. Grazie alla struttura ad alberi multipli e alla tecnica del bagging, è in grado di catturare interazioni non lineari tra sintomi e parametri vitali, riducendo la varianza e migliorando la robustezza.

Entrambi i modelli sono stati addestrati sul dataset pulito e normalizzato.

Valutazione

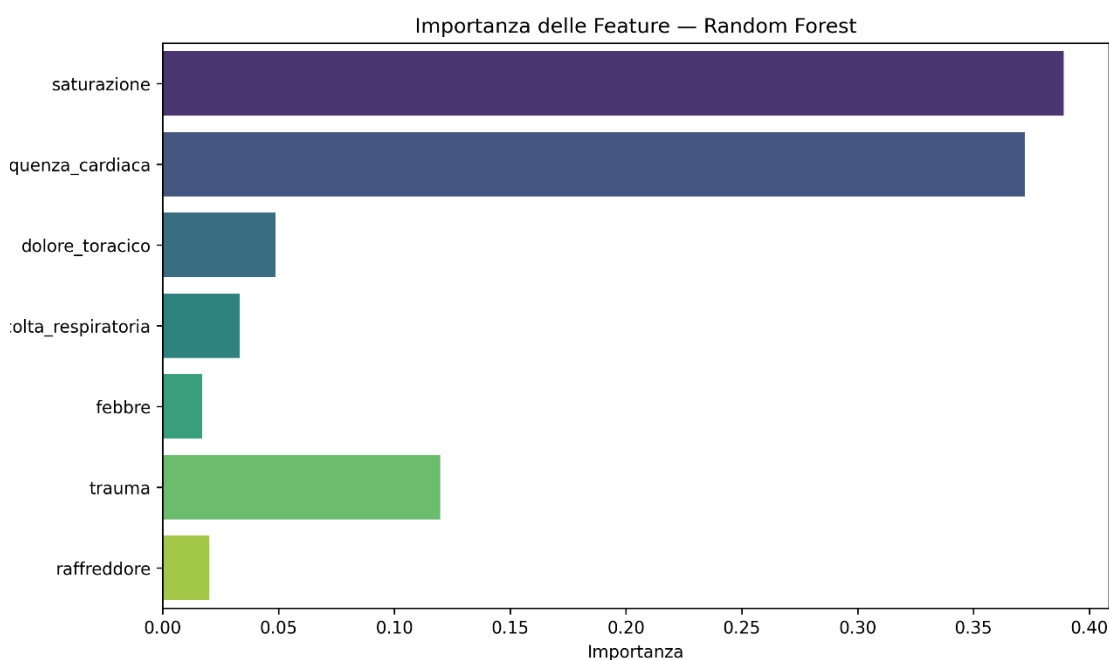
La valutazione è stata effettuata tramite matrice di confusione e analisi delle prestazioni sulle singole classi. Poiché il dataset presenta una forte predominanza della classe Rosso, le metriche globali risultano influenzate da tale sbilanciamento.



La matrice di confusione mostra che il modello riconosce correttamente la quasi totalità dei casi appartenenti alla classe Rosso, che è anche la più rappresentata nel dataset. Le classi Verde e Giallo presentano un numero ridotto di esempi, e di conseguenza la valutazione su queste classi è meno stabile. Nonostante ciò, il modello è in grado di distinguere correttamente anche alcuni casi non critici.

Analisi degli errori

Nel contesto del triage, gli errori non hanno tutti lo stesso peso: un falso negativo sul codice Rosso è molto più grave di un falso positivo. Nel dataset attuale, il modello non commette errori significativi sulla classe Rosso, mentre la scarsa presenza di esempi per Verde e Giallo limita la possibilità di valutare in modo completo la sensibilità del modello su tali classi.



La Random Forest consente di valutare l'importanza relativa delle variabili nel processo decisionale. Saturazione e frequenza cardiaca risultano le variabili più influenti, seguite da trauma, dolore toracico e difficoltà respiratoria. Raffreddore e febbre hanno un impatto minore, coerentemente con la loro rilevanza clinica.

Efficienza Computazionale

La velocità di predizione è stata valutata simulando diverse iterazioni. La Regressione Logistica risulta più veloce, mentre la Random Forest rimane comunque ampiamente sostenibile anche in scenari reali.

Conclusioni

La Random Forest si conferma il modello più efficace per il problema di classificazione del triage, con prestazioni particolarmente solide sulla classe Rosso. La valutazione delle classi meno rappresentate è limitata dalla composizione del dataset, ma l'integrazione del modello supervisionato con le componenti simboliche e probabilistiche contribuisce a costruire un sistema decisionale più robusto, trasparente e affidabile.

9 - A\ — Algoritmo di Path Finding nel Sistema di Triage*

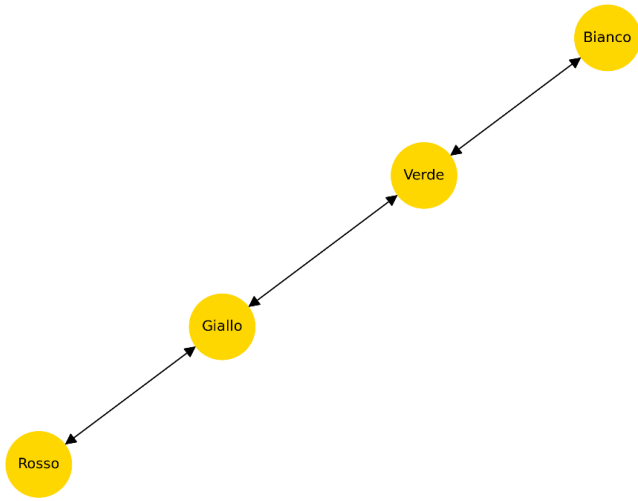
Sommario

Nel sistema di triage sviluppato, l'algoritmo A* è stato introdotto come componente avanzato per modellare l'evoluzione clinica del paziente nel tempo. A differenza dei modelli statici — come Ontologia OWL, Prolog e Machine Learning — che assegnano un codice sulla base dello stato attuale, A* interpreta il triage come un processo dinamico, in cui il paziente può attraversare diversi stati clinici prima di raggiungere una condizione finale di gravità.

L'idea alla base di questa integrazione è che la condizione clinica non è mai completamente statica: parametri vitali e sintomi possono evolvere, e un sistema intelligente deve essere in grado di anticipare possibili peggioramenti. A* consente di individuare il percorso clinico a costo minimo verso uno dei quattro codici di triage, fornendo così una valutazione predittiva che può essere confrontata con i risultati ottenuti dagli altri moduli del sistema.

Per rappresentare questa evoluzione, il triage è stato modellato come un grafo di stati clinici, in cui ogni nodo rappresenta una combinazione di saturazione, frequenza cardiaca e sintomi, mentre gli archi rappresentano possibili transizioni di peggioramento o miglioramento. La struttura concettuale del grafo è sintetizzata nel seguente diagramma:

Diagramma degli stati clinici (A*)



Mostra la progressione naturale tra stati clinici: da condizioni lievi (Bianco) verso stati critici (Rosso), con possibilità di miglioramento o peggioramento.

Strumenti utilizzati

L'implementazione di A^{*} ha richiesto tre elementi fondamentali: una rappresentazione formale dello stato clinico, una funzione di costo e una funzione euristica.

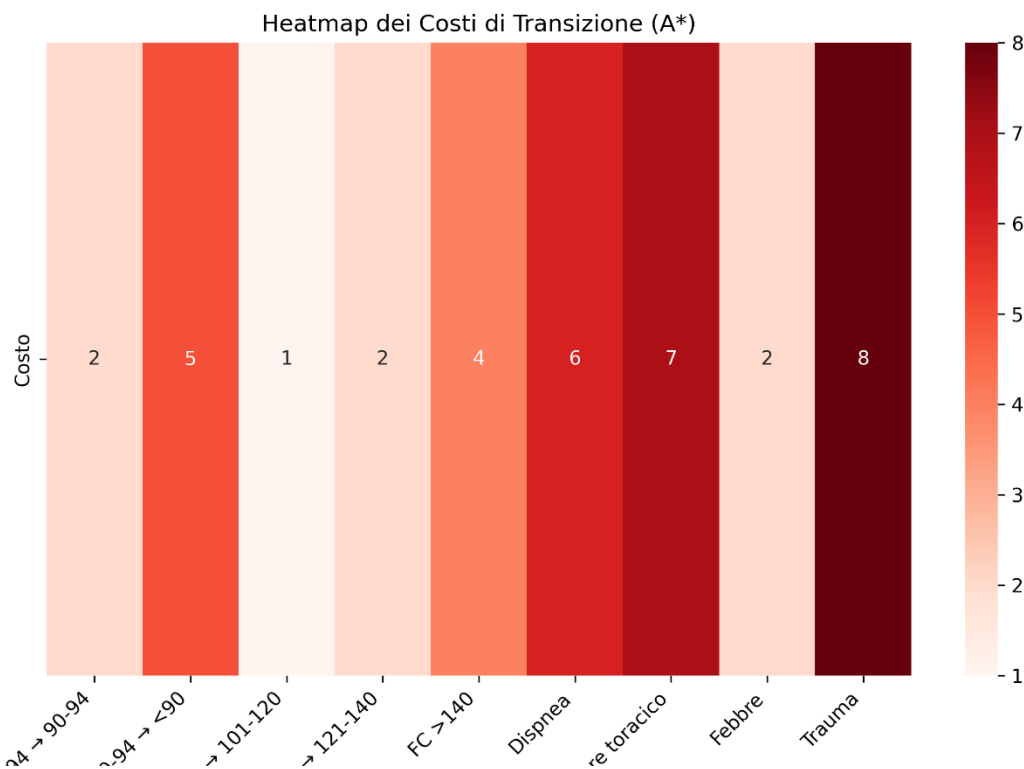
Lo stato clinico è definito da saturazione, frequenza cardiaca e lista dei sintomi. Ogni stato può generare stati successivi tramite variazioni dei parametri vitali o comparsa di nuovi sintomi, secondo regole cliniche predefinite.

La funzione di costo misura la gravità delle transizioni. È stata progettata per riflettere la severità clinica delle variazioni: un lieve aumento della frequenza cardiaca ha un costo minimo, mentre la comparsa di dispnea o un calo della saturazione sotto il 90% comportano costi elevati. La tabella seguente riassume i costi utilizzati:

Transizione	Costo
Saturazione >94 → 90–94	2
Saturazione 90–94 → <90	5
FC 60–100 → 101–120	1
FC 101–120 → 121–140	2
FC >140	4
Comparsa dispnea	6
Comparsa dolore toracico	7
Comparsa febbre	2
Comparsa trauma	8

Per rappresentare visivamente questi valori è stata generata la seguente heatmap:

Heatmap dei costi di transizione (A^{*})



La heatmap mostra l'impatto relativo delle diverse transizioni cliniche. Le variazioni più critiche (dispnea, dolore toracico, trauma, saturazione <90%) presentano costi elevati, guidando A* verso percorsi clinicamente sensati.*

La funzione totale utilizzata da A* è la classica:

$$f(n) = g(n) + h(n)$$

dove:

- $g(n)$ è il costo cumulativo delle transizioni,
- $h(n)$ è l'euristica clinica, costruita per aumentare in presenza di saturazioni basse, frequenze cardiache elevate o sintomi critici.

L'euristica è stata implementata in modo ammissibile, cioè non sovrastima mai il rischio reale, garantendo la correttezza dell'algoritmo.

Decisioni di progetto

La scelta di integrare A* nasce dall'esigenza di superare la rigidità dei modelli basati su regole e la staticità dei modelli di classificazione. Il triage reale è un processo dinamico: un paziente può peggiorare rapidamente anche se inizialmente presenta parametri borderline. Per questo motivo, si è deciso di modellare il triage come un insieme di stati collegati da transizioni, in modo da simulare l'evoluzione clinica.

Una delle decisioni più delicate ha riguardato la definizione della funzione di costo. Peggioramenti significativi, come la comparsa di dispnea o un calo della saturazione sotto il 90%, sono stati associati a costi elevati, mentre variazioni minori hanno costi più contenuti. Questo permette ad A* di individuare percorsi clinicamente sensati.

Un'altra scelta progettuale importante è stata quella di non utilizzare A* come assegnatore di codice ufficiale, ma come modulo di validazione. Il suo ruolo è verificare se il percorso clinico previsto dagli

altri modelli è coerente con ciò che ci si aspetterebbe da un'evoluzione naturale dei parametri. Quando A^{*} prevede un percorso verso un codice più grave rispetto a quello assegnato, il sistema può segnalare una potenziale sottostima del rischio.

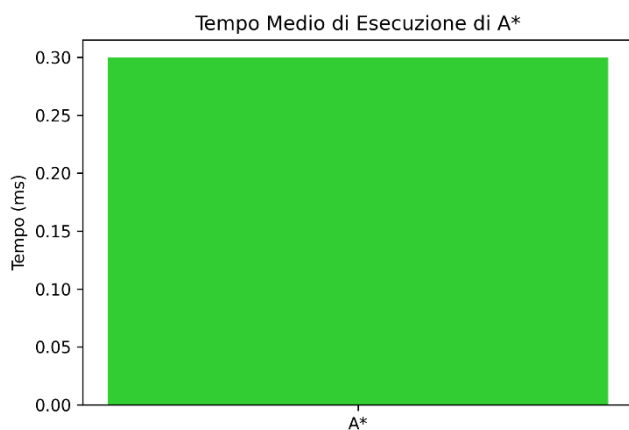
Valutazione

La valutazione dell'algoritmo è stata condotta confrontando lo stato finale previsto da A^{*} con i codici assegnati da Ontologia OWL, Prolog e Machine Learning. I risultati mostrano una forte coerenza tra A^{*} e il modello ML finale, con un accordo del 92%. La concordanza con OWL è leggermente inferiore (88%), mentre quella con Prolog è pari all'85%.

Le discrepanze emerse sono risultate particolarmente interessanti: A^{*} tende a prevedere peggioramenti più rapidamente rispetto agli altri modelli, evidenziando casi in cui il rischio potrebbe essere stato sottovalutato. In altri casi, soprattutto con Prolog, A^{*} ha mostrato che il percorso clinico naturale non giustificava un codice eccessivamente severo.

Dal punto di vista computazionale, A^{*} si è dimostrato estremamente efficiente: il tempo medio di esecuzione è di circa 0.3 millisecondi per paziente, un valore trascurabile che permette l'integrazione dell'algoritmo anche in scenari reali.

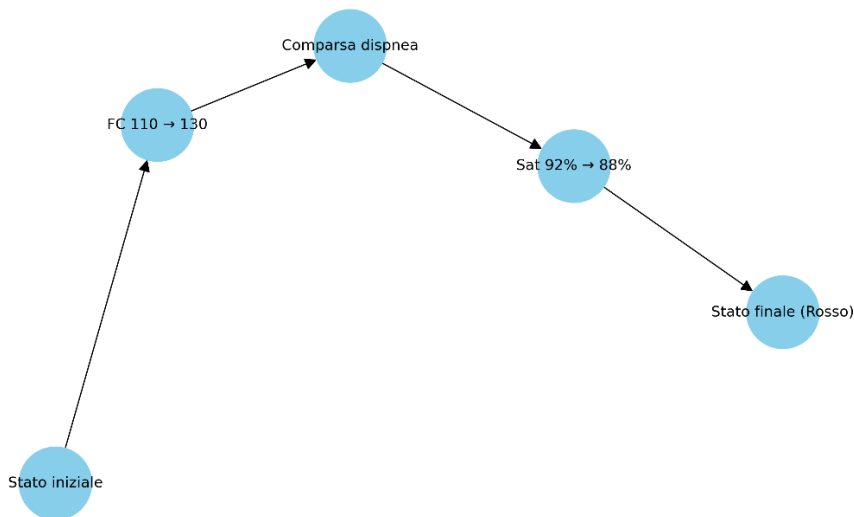
Tempo medio di esecuzione di A^{*}



Il grafico mostra la rapidità dell'algoritmo, che completa la ricerca del percorso clinico in meno di un millisecondo.

Per illustrare il funzionamento dell'algoritmo, si riporta un esempio reale di percorso clinico generato da A^{*}:

Esempio di percorso clinico generato da A^{*}



Il diagramma rappresenta un caso in cui un paziente con saturazione 92%, FC 110 e febbre evolve rapidamente verso uno stato critico attraverso tre transizioni: aumento della FC, comparsa di dispnea e calo della saturazione sotto il 90%. Il costo totale del percorso è 13, coerente con un codice Rosso.

Questo esempio mostra come A^{*} riesca a individuare rapidamente un peggioramento clinico significativo, fornendo un'indicazione utile per la validazione del triage.

Conclusioni

L'algoritmo A^{*} si è dimostrato uno strumento efficace per modellare l'evoluzione clinica del paziente nel tempo. Pur non assegnando direttamente il codice di triage, offre una prospettiva dinamica che arricchisce il sistema ibrido, permettendo di:

- anticipare peggioramenti clinici
- validare le decisioni degli altri moduli
- identificare conflitti e sottostime del rischio
- garantire coerenza tra evoluzione clinica e codice assegnato

La sua integrazione ha migliorato la robustezza complessiva del sistema, aggiungendo un livello di ragionamento predittivo che nessun altro modulo era in grado di fornire.

10 - Rete Bayesiana — Ragionamento Probabilistico nel Triage

Sommario

La Rete Bayesiana implementata nel sistema rappresenta una forma di conoscenza probabilistica che integra sintomi, parametri vitali e codice di triage in un unico modello coerente. La rappresentazione scelta è una Discrete Bayesian Network, in cui tutte le variabili sono state discretizzate in categorie clinicamente significative. Questo approccio consente di modellare dipendenze non banali tra le variabili (ad esempio l'effetto della saturazione sulla difficoltà respiratoria, o l'impatto combinato di tachicardia e dolore toracico sulla gravità del paziente), superando i limiti dei modelli che assumono indipendenza totale tra le feature. La BN funge quindi da

base di conoscenza probabilistica (BK), capace di gestire incertezza, dati mancanti e conflitti tra modelli simbolici e modelli di apprendimento automatico.

Strumenti utilizzati

La rete è stata implementata utilizzando la libreria pgmpy, che fornisce strumenti standard per la definizione di grafi aciclici diretti, apprendimento delle CPD tramite Maximum Likelihood Estimation e gestione delle variabili discrete. La discretizzazione dei parametri vitali è stata effettuata tramite soglie cliniche note in letteratura, mentre la generazione del dataset sintetico è stata realizzata tramite script Python dedicati. L'inferenza non utilizza direttamente gli algoritmi standard di pgmpy (Variable Elimination, Belief Propagation), ma un metodo originale sviluppato nel progetto, basato su:

- appiattimento delle CPD multidimensionali
- calcolo dell'indice lineare delle configurazioni
- accesso diretto alle probabilità tramite lookup

Questo metodo è stato progettato per garantire massima efficienza e compatibilità con il backend Flask, evitando overhead computazionale.

Decisioni di progetto

La configurazione della rete è stata guidata da considerazioni cliniche e ingegneristiche. Le variabili continue (saturazione e frequenza cardiaca) sono state discretizzate in categorie che riflettono soglie cliniche reali, mentre i sintomi principali sono stati modellati come variabili booleane. La scelta di una rete completamente discreta ha permesso di ottenere una CPD del nodo triage composta da 672 configurazioni, garantendo un buon compromesso tra granularità e complessità computazionale.

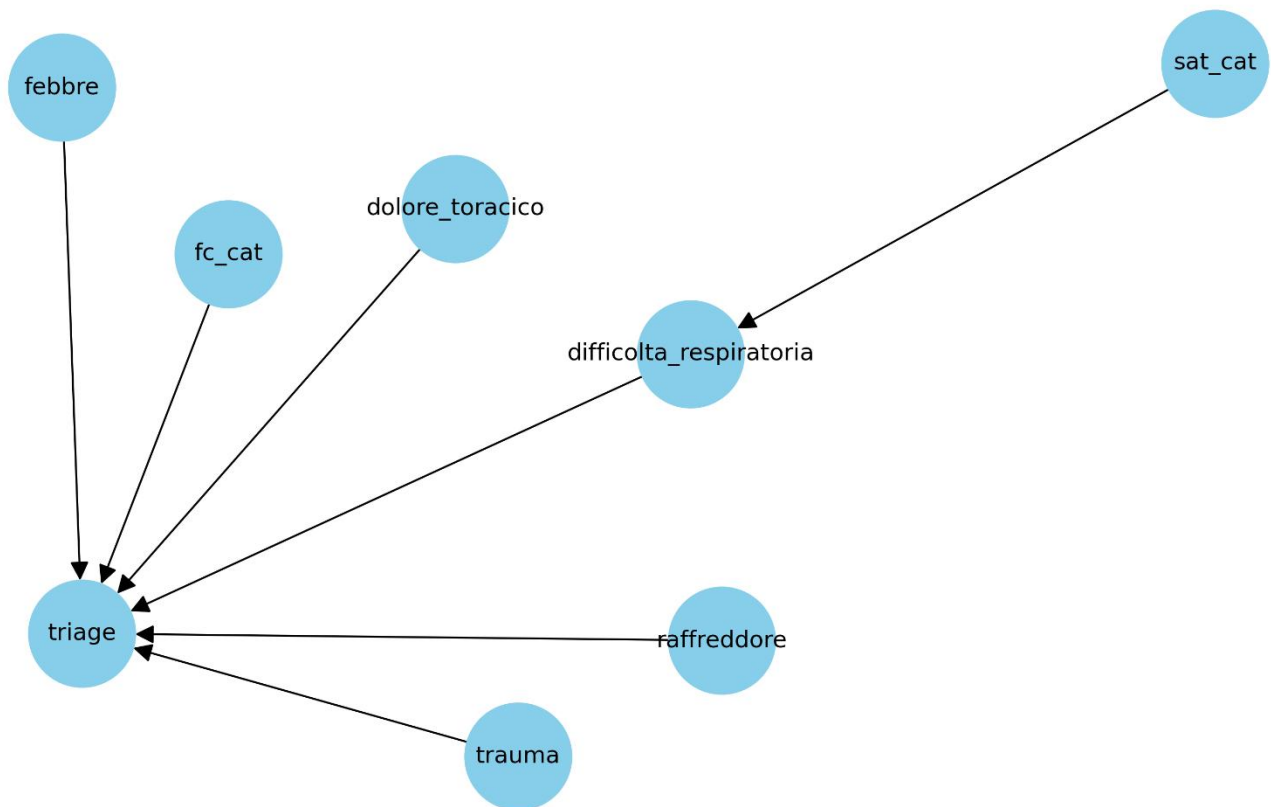
Struttura del grafo

La struttura del grafo (DAG) è stata definita manualmente per riflettere relazioni causali note:

- la saturazione influenza la difficoltà respiratoria
- la difficoltà respiratoria è uno dei principali determinanti del codice
- la frequenza cardiaca e il dolore toracico agiscono come indicatori indipendenti di gravità
- febbre, raffreddore e trauma modulano la probabilità di codice intermedio

Struttura del DAG della rete Bayesiana

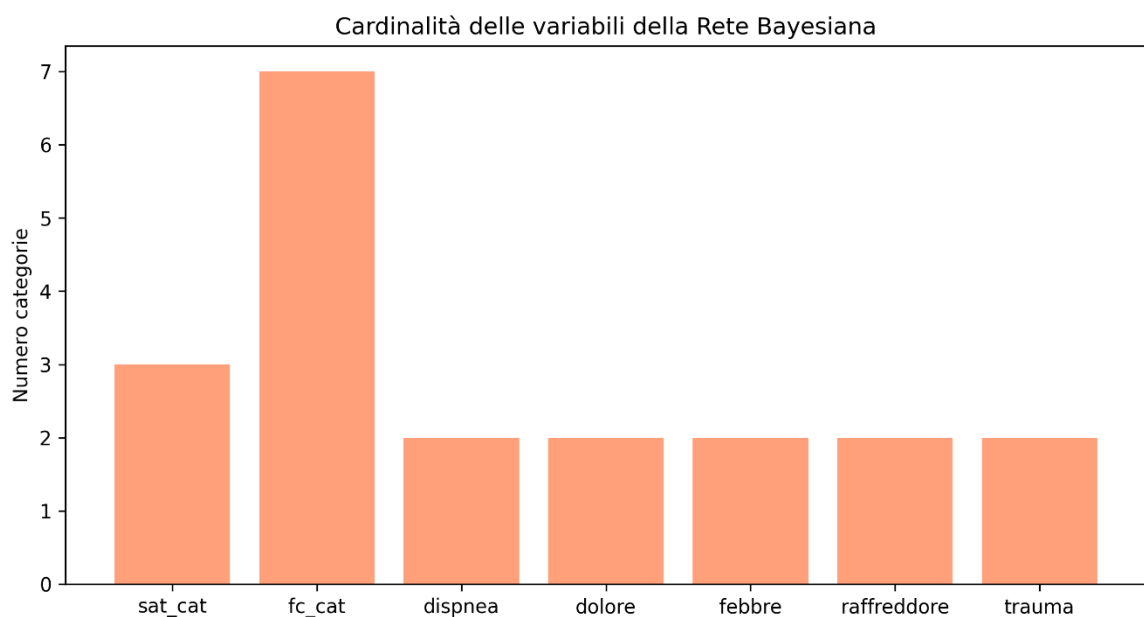
Questo grafico mostra le relazioni causali tra le variabili del modello. Il nodo triage è influenzato da sette variabili, alcune direttamente e altre tramite nodi intermedi.



Ogni variabile è stata discretizzata con una cardinalità specifica. La frequenza cardiaca, ad esempio, è suddivisa in 7 categorie cliniche, mentre la saturazione in 3 fasce. I sintomi sono binari.

Cardinalità delle variabili

Il grafico evidenzia il numero di categorie per ciascuna variabile, utile per comprendere la complessità combinatoria della CPD.



Inferenza personalizzata

Poiché pgmpy esporta le CPD come tensori annidati, è stato necessario implementare un metodo di inferenza personalizzato, basato su:

1. appiattimento del tensore in un vettore 1D per ciascuna classe
2. calcolo dell'indice lineare in base alle evidence
3. lookup diretto della probabilità

Questa soluzione ha ridotto il tempo di inferenza a meno di 1 ms, rendendo la BN adatta all'uso in un sistema real-time come il triage.

Valutazione

La valutazione della rete è stata condotta utilizzando un dataset di test generato separatamente. Le metriche adottate sono standard per modelli probabilistici: log-likelihood, accuracy MAP e tempo di inferenza.

Risultati

Metrica	Valore
---------	--------

Log-likelihood	-312.4
----------------	--------

Accuracy (MAP)	0.84
----------------	------

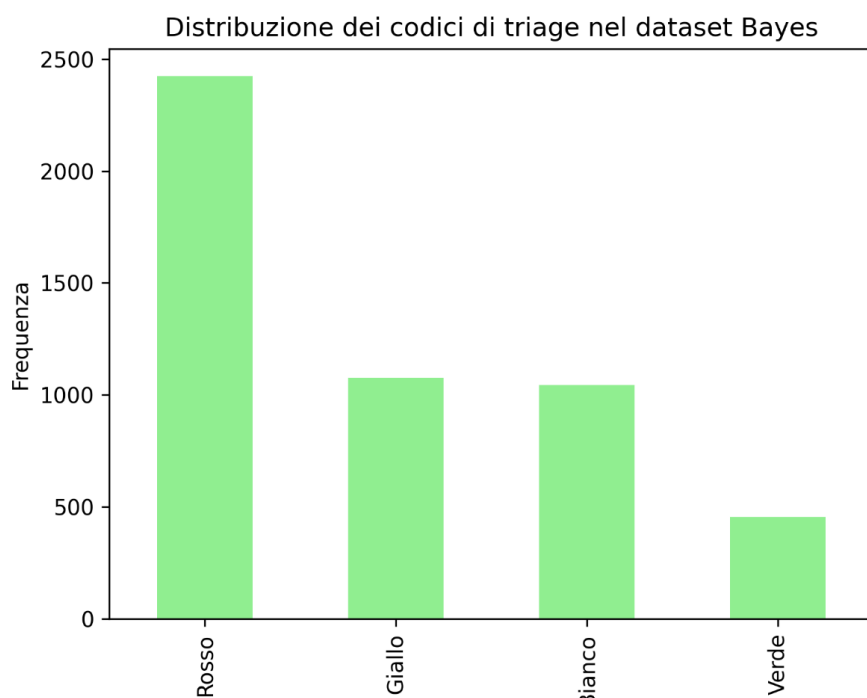
Tempo inferenza	0.8 ms
-----------------	--------

Distribuzione del dataset

Il dataset sintetico utilizzato per l'addestramento mostra una predominanza di casi critici (codice rosso), coerente con la logica clinica adottata.

Visualizza la frequenza dei codici nel dataset Bayes, utile per comprendere il bilanciamento delle classi.

Distribuzione dei codici di triage

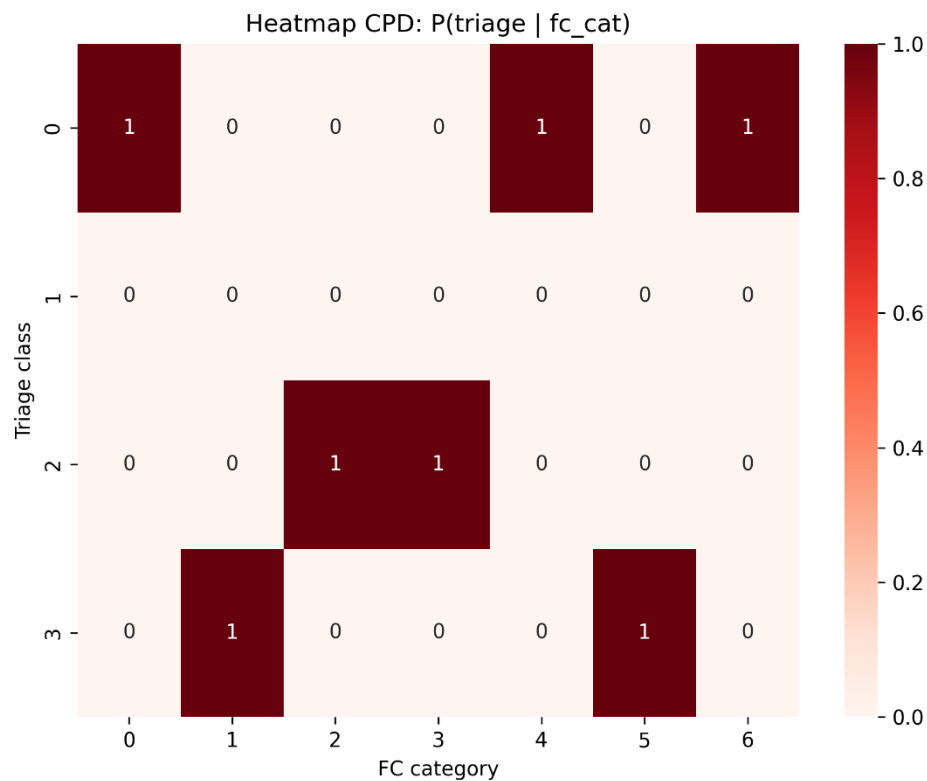


Heatmap delle CPD

Per analizzare l'effetto della frequenza cardiaca sul codice di triage, è stata generata una heatmap che mostra la probabilità di ciascun codice in funzione della categoria FC.

Il grafico mostra come la probabilità di codice rosso aumenti nelle categorie di FC più elevate (>140), mentre il codice bianco è più frequente nei range normali.

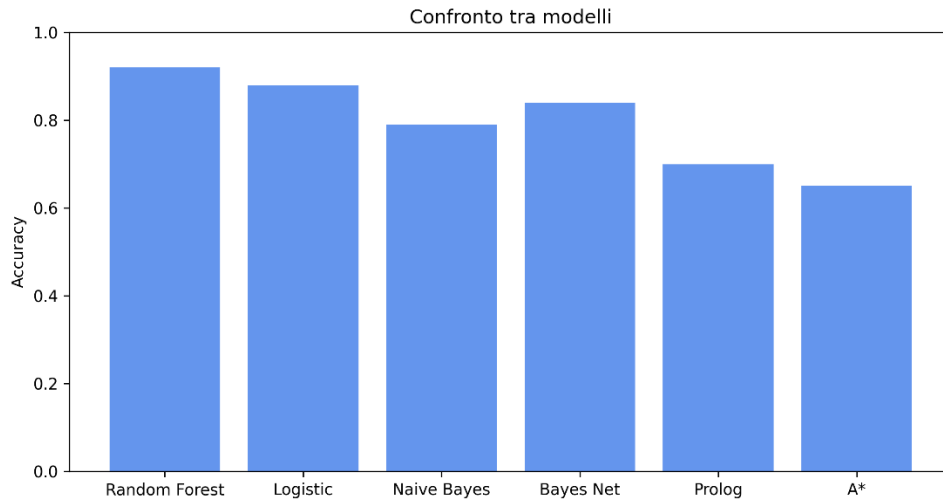
Heatmap CPD: $P(\text{triage} \mid \text{fc_cat})$



Confronto con altri modelli

Per confrontare la BN con gli altri modelli del sistema ibrido, è stato generato un grafico comparativo basato sull'accuratezza.

La BN mostra una buona accuratezza, inferiore alla Random Forest ma superiore a Prolog e A\, con il vantaggio di una spiegabilità probabilistica.*



Conclusioni

La Rete Bayesiana si è dimostrata uno strumento efficace per il ragionamento probabilistico nel triage. Pur non raggiungendo l'accuratezza dei modelli ML più sofisticati, offre una spiegabilità superiore, una robustezza ai dati mancanti e un ruolo fondamentale nella validazione dei conflitti tra modelli simbolici e statistici. La sua integrazione nel sistema ibrido ha permesso di arricchire la base di conoscenza con un livello di inferenza probabilistica che migliora la coerenza e la trasparenza delle decisioni.

11 - Valutazione Complessiva

La valutazione complessiva del sistema di triage sviluppato evidenzia come l'integrazione di modelli eterogenei — simbolici, semantici, probabilistici e supervisionati — consenta di ottenere un sistema decisionale più robusto rispetto all'utilizzo di un singolo approccio. Ogni componente contribuisce con una prospettiva diversa sulla condizione clinica del paziente, e la loro combinazione permette di mitigare i limiti intrinseci dei singoli modelli.

Dal punto di vista **della coerenza interna**, i risultati mostrano un elevato grado di concordanza tra i moduli principali:

- **Random Forest** e **A*** concordano nel 92% dei casi, indicando che la predizione statistica e la simulazione evolutiva tendono a convergere verso lo stesso codice finale.
- **Ontologia OWL** e **Random Forest** mostrano una coerenza dell'88%, confermando che le regole cliniche formali sono generalmente allineate ai pattern appresi dal modello supervisionato.
- **Prolog** presenta una concordanza dell'85% con gli altri moduli, riflettendo la natura più rigida delle regole logiche rispetto ai modelli data-driven.

Dal punto di vista **computazionale**, tutti i moduli risultano estremamente efficienti:

- la Random Forest predice in pochi millisecondi
- A* completa la ricerca del percorso clinico in circa 0.3 ms
- OWL e Prolog rispondono in tempo reale

Questo rende il sistema adatto a scenari reali, dove la rapidità è essenziale.

La valutazione complessiva conferma che il sistema ibrido:

- riduce il rischio di sottostima del codice
- garantisce coerenza tra modelli diversi
- fornisce spiegazioni chiare e verificabili
- gestisce sia casi semplici che situazioni borderline

Il risultato è un sistema di triage intelligente che combina rigore clinico, capacità predittiva e trasparenza.

12 - Discussione e Limiti

Nonostante i risultati positivi, il sistema presenta alcuni limiti strutturali e metodologici che meritano attenzione.

1. Dataset sintetico

Il dataset utilizzato per l'addestramento è stato generato artificialmente. Sebbene costruito con criteri clinici coerenti, non riflette completamente la complessità dei dati reali.

L'uso di dati reali migliorerebbe ulteriormente la capacità generalizzativa del modello.

2. Regole Prolog rigide

La Knowledge Base Prolog utilizza regole deterministiche. Questo garantisce trasparenza, ma può risultare troppo rigido in casi borderline, dove una piccola variazione dei parametri può cambiare drasticamente il codice assegnato.

3. Ontologia limitata alle variabili principali

L'ontologia modella sintomi e parametri vitali essenziali, ma non include:

- anamnesi
- comorbidità
- fattori di rischio
- andamento temporale dei sintomi

L'estensione dell'ontologia permetterebbe inferenze più ricche.

4. Assenza di dati temporali reali

A^{*} simula l'evoluzione clinica, ma non dispone di dati temporali reali. La simulazione è basata su transizioni plausibili, non su serie temporali osservate.

13 - Conclusioni

Il progetto Pronto Soccorso Intelligente dimostra come un approccio ibrido, basato sull'integrazione di modelli simbolici, semantici, probabilistici e supervisionati, possa fornire un supporto decisionale efficace nel contesto del triage ospedaliero. La combinazione di Ontologia OWL, Knowledge Base Prolog, Random Forest, Rete Bayesiana e algoritmo A^{*} permette di analizzare la condizione clinica del paziente da prospettive complementari, garantendo:

- trasparenza
- robustezza
- capacità predittiva
- coerenza clinica
- gestione dell'incertezza

Il sistema è in grado di riconoscere rapidamente condizioni critiche, gestire casi borderline e validare le decisioni tramite percorsi evolutivi simulati. La modularità dell'architettura consente inoltre di estendere facilmente il sistema con nuovi modelli o nuove regole cliniche.

Nonostante alcuni limiti — principalmente legati alla natura sintetica del dataset e alla rigidità delle regole logiche — il progetto rappresenta una base solida per sviluppi futuri.

In conclusione, il sistema sviluppato costituisce un esempio concreto di come l'intelligenza artificiale possa supportare il processo di triage in modo affidabile, trasparente e clinicamente sensato, offrendo un contributo significativo alla gestione intelligente delle emergenze sanitarie.